Dominio 4 spiegato da zero: prompt engineering e output strutturato (20%)

Il **prompt** è il testo che scrivi per dire a Claude cosa fare. Il **prompt engineering** è l'arte di scriverlo bene. Questo dominio si concentra su tre cose: come scrivere **criteri** che Claude applica in modo coerente, come ottenere **dati in formato preciso** (per esempio un modulo compilato) e come gestire **lavori in serie, revisioni e costi**.

Il filo conduttore da tenere sempre in mente: **lo schema garantisce la forma dei dati, non la loro correttezza.**

> **Prima di iniziare.** Quando incontri → **Pre-corso** trovi il link alla spiegazione completa. Per questo dominio conviene aver letto almeno:
>
> - JSON e JSON Schema (il "modulo" che Claude deve compilare);
> - Prompt di sistema e few-shot e XML e tag;
> - Sincrono, asincrono e batch;
> - Token e finestra di contesto e Cache;
> - PDF e documenti non strutturati (per gli scenari di estrazione dati).
>
> Per i termini specifici di Claude tieni aperto il Vocabolario.

---

1. Criteri espliciti: scrivere regole che si possano applicare

Immagina di dare a un collega il compito di segnalare "i contratti rischiosi". Se gli dici "segnala con prudenza" o "usa il tuo giudizio", ogni giorno segnalerà cose diverse. Non perché sbagli, ma perché la regola è troppo vaga.

Per ogni tipo di segnalazione servono tre elementi:

- **Cosa rientra**: le condizioni precise che la attivano.
- **Cosa resta escluso**: i casi limite che non devono attivarla.
- **Esempi di confine**: coppie concrete che mostrano dove passa il limite.

Un esempio: da vago a esplicito

Vago: "*Segnala le clausole rischiose nei contratti.*"

Esplicito:

```
<criterio nome="penale_sproporzionata">
Rientra: penali per ritardo superiori al 10% del valore del contratto,
oppure penali senza un tetto massimo.
Escluso: penali fino al 10% con un tetto massimo indicato;
penali a carico nostro verso il fornitore.
Esempio che rientra: "1% al giorno di ritardo, senza limite".
```

```
Esempio escluso: "0,5% a settimana, fino a un massimo del 5%".
</criterio>
```

→ Pre-corso: XML e tag. I tag come `<criterio>` servono a separare chiaramente ogni regola dal resto del prompt.

Con il criterio esplicito, due persone diverse (o Claude in due giorni diversi) arrivano alla stessa decisione.

Il rischio dei falsi positivi

Un **falso positivo** è una segnalazione sbagliata: il sistema dice "qui c'è un problema" e invece non c'è. Un **falso negativo** è il contrario: un problema vero che non viene segnalato.

Se il sistema segnala troppe cose inutili, la gente smette di leggere le segnalazioni, anche quelle importanti. Un allarme che suona sempre viene ignorato. Se una categoria è rumorosa, si disattiva per un po', si migliorano i criteri, poi si riattiva.

Nota bene: chiedere a Claude di "essere più accurato" o "segnalare solo i problemi importanti" **non** riduce i falsi positivi in modo affidabile. Servono criteri più precisi.

---

2. Few-shot: insegnare con degli esempi

Il **few-shot** significa mostrare a Claude alcuni esempi completi prima di chiedergli un lavoro. È come dare a un nuovo assunto tre pratiche già risolte.

→ Pre-corso: Prompt di sistema e few-shot, con un esempio completo di prompt.

Quando serve:

- scenari ambigui;
- quando devi mostrare un formato preciso;
- per i casi limite;
- per ridurre gli errori nell'estrazione di dati.

Regole:

- **Da 2 a 4 esempi mirati.** Con uno solo non si capisce la regola. Con troppi si riempie inutilmente la memoria di lavoro (il contesto) e Claude finisce per copiare gli esempi invece di capire il modello.
- **Ogni esempio mostra input, output e ragionamento**: il perché la risposta è giusta. Così Claude impara il principio e lo applica a casi nuovi.

Un esempio con il ragionamento:

```
<esempio>
<email>Il pacco è arrivato aperto e manca un pezzo. Vorrei anche cambiare
l'indirizzo per i prossimi ordini.</email>
<categoria>reclamo_spedizione</categoria>
<ragionamento>Ci sono due richieste. Il pezzo mancante è un problema da risolvere
subito; il cambio di indirizzo è una richiesta secondaria. La categoria
```

```
segue il problema principale.</ragionamento>
</esempio>
```

Scegli esempi **diversi tra loro** e che coprano proprio i casi difficili. Se tutti gli esempi sono email brevi e cortesi, Claude non saprà cosa fare con un'email lunga e arrabbiata.

---

3. Output strutturato: far compilare a Claude un modulo

Il problema del testo libero è che, se chiedi a Claude di restituire dati, può scrivere in modi diversi: un JSON (un formato a campi e valori, tipo `"totale": 150`) rotto, un campo dimenticato, un nome di campo diverso, un numero scritto come testo.

→ Pre-corso: JSON

Perché è un problema? Perché i dati estratti di solito non li legge una persona, ma un **programma** (il gestionale, il database, il sistema contabile). Una persona capisce che "centocinquanta euro" e `150` sono la stessa cosa; un programma no. Un JSON con una virgola fuori posto fa fallire l'intera elaborazione.

La soluzione è **costringerlo a un modulo**: uno schema che dice quali campi ci sono, di che tipo sono, quali sono obbligatori.

→ Pre-corso: JSON Schema

Un esempio di schema per una fattura:

```
{
  "type": "object",
  "properties": {
    "fornitore": { "type": "string" },
    "numero_fattura": { "type": "string" },
    "data": { "type": "string" },
    "totale": { "type": "number" },
    "valuta": { "type": "string", "enum": ["EUR", "USD", "other"] },
    "numero_ordine": { "type": ["string", "null"] }
  },
  "required": ["fornitore", "numero_fattura", "data", "totale", "valuta",
"numero_ordine"],
  "additionalProperties": false
}
```

Ci sono tre modi per farlo:

Soluzione	Come funziona	Nota per l'esame
1. `tool_use` con schema	Si definisce uno strumento il cui "input" è il modulo, e si obbliga Claude a usarlo (`tool_choice` forzato)	Non accettato da Fable 5.1 e Opus 5.5

Soluzione	Come funziona	Nota per l'esame
2. `strict`	Si aggiunge `"strict": true` alla definizione dello strumento. Richiede `additionalProperties: false` e tutte le proprietà in `required`	Garantisce che la forma rispetti lo schema
3. `output_config.format`	Si chiede a Claude di rispondere direttamente in JSON secondo lo schema	Il modo moderno. Il vecchio campo `output_format` è deprecato

Il primo metodo è un "trucco" storico: si finge che il modulo sia l'input di uno strumento, così Claude è costretto a compilarlo. Il terzo è il modo diretto: si chiede la risposta già nel formato giusto.

La distinzione più importante dell'esame

Lo schema garantisce la **conformità**: i tipi giusti, i campi obbligatori, i valori ammessi.

Ma non garantisce la **correttezza**. Immagina una fattura dove il totale scritto è 150 euro, ma la somma delle voci fa 175. Il JSON è perfettamente valido: campi giusti, numeri al posto giusto. Il dato però è sbagliato. Solo il codice può accorgersene, controllando che la somma torni.

Frase da ricordare: la struttura è garantita dallo schema, la verità dal codice.

Un'analogia: un modulo dell'anagrafe può obbligarti a scrivere una data nel campo "data di nascita" e a non lasciarlo vuoto. Non può verificare che quella sia davvero la tua data di nascita.

→ Pre-corso: Allucinazioni. Uno schema tutto obbligatorio può addirittura **spingere** Claude a inventare un valore pur di riempire il campo.

Progettare bene lo schema

- **Campi nullable** (`["string", "null"]`): usali quando un'informazione può semplicemente non esserci. Se tutto è obbligatorio, Claude tende a inventare pur di riempire il modulo.
- **Enum con `"other"`**: un'opzione "altro" per i casi non previsti, con un campo di testo per descriverli.
- **Enum con `"unclear"`**: un'opzione "non è chiaro", così Claude non deve indovinare.

Nell'esempio di schema sopra, `numero_ordine` è nullable (molte fatture non lo riportano) e `valuta` ha l'opzione `"other"` (per le valute non previste).

I limiti dello `strict`

La modalità strict non supporta alcuni vincoli: `minimum` e `maximum` (valori numerici minimi e massimi), `minLength` e `maxLength` (lunghezza minima e massima del testo), `pattern` (formati tipo "tre lettere, trattino, quattro cifre"), `oneOf` e schemi ricorsivi. Questi controlli vanno fatti nel codice, dopo la risposta.

→ Pre-corso: Regex. Un `pattern` è un'espressione regolare: se lo schema strict non lo supporta, la regex si applica nel codice dopo aver ricevuto la risposta.

---

4. Il ciclo di convalida: quando riprovare e quando no

Il ciclo è: Claude genera la risposta, il codice la controlla, se ci sono errori si aggiungono al prompt gli errori precisi e si riprova, fino a un massimo di tentativi.

```
Claude estrae i dati
      ↓
Il codice controlla (schema + regole di coerenza)
      ↓
Tutto ok? → salva
Errori?   → rimanda a Claude il documento + la risposta + l'errore preciso
            ("il totale 150 non corrisponde alla somma delle voci, 175")
            → nuovo tentativo, fino a un massimo
```

Ma attenzione: si controllano due tipi di problemi.

- **Convalida dello schema** (affidabile): campo mancante, tipo sbagliato, valore fuori dall'elenco, data scritta male.
- **Convalida semantica** (serve codice personalizzato): somma che non torna, data nel futuro, riferimenti che non esistono, contraddizioni tra campi.

Il messaggio di errore deve essere **specifico**. "Riprova, c'è un errore" non aiuta; "il campo `data` contiene 31/02/2026, che non esiste" sì.

Trappola decisiva: riprovare non serve se l'informazione **non è nel documento**. Se una fattura non ha il numero d'ordine, rifarla a Claude non lo farà comparire. La risposta giusta è accettare `null`, cioè "non presente".

→ Pre-corso: OCR. C'è un altro caso in cui riprovare non serve: se il testo di partenza è già sbagliato (una scansione letta male), nessun nuovo tentativo lo correggerà.

Per capire quali regole scattano troppo spesso, si può aggiungere un campo `detected_pattern`: indica quale parte del testo ha fatto scattare il risultato. Così vedi i falsi positivi ricorrenti.

---

5. Batch: lavorare in serie e risparmiare

→ Pre-corso: Sincrono, asincrono e batch

Abbiamo già visto il Batch nel dominio 3. Riassumiamolo qui con l'analogia della lavanderia.

- **Sincrono**: porti i vestiti e aspetti in negozio. Serve per le urgenze: revisione prima di un'approvazione, risposta in tempo reale.
- **Batch**: porti una montagna di vestiti, li ritiri domani. Costa il 50% in meno, ma non sai esattamente quando. Va bene per analisi notturne, report settimanali, generazione di test.

Regole pratiche:

- **Costo**: -50%. **Tempo**: fino a 24 ore, senza garanzie precise.

- **Correlazione**: i risultati tornano in ordine qualsiasi. Per riconoscere quale risposta appartiene a quale richiesta usi `custom_id`, l'etichetta che hai assegnato, mai la posizione nella lista.
- **Più turni**: un batch non supporta conversazioni a più passaggi. Ogni richiesta deve essere autosufficiente.
- **Errori**: se 50 richieste su 1.000 falliscono, reinvii **solo quelle 50** (identificate dal `custom_id`), non tutto il lotto.
- **Prima di scalare**: prova il prompt su un piccolo campione. Meglio scoprire un errore su 20 documenti che su 10.000.

Un esempio di `custom_id` ben scelto: `fattura-2026-000123`. Quando il risultato torna, sai subito a quale documento si riferisce e puoi salvarlo nel posto giusto, indipendentemente dall'ordine di arrivo.

"Più turni" significa anche: niente ciclo con gli strumenti. Se per rispondere Claude deve chiedere uno strumento e ricevere il risultato, quella richiesta non può stare in un batch.

---

6. Revisione in più passaggi

Qui torna il tema del dominio 3: chi ha scritto il codice non lo rivede bene. La stessa sessione conserva il ragionamento che ha portato alle decisioni e tende a giustificarle. La revisione va quindi fatta in una **sessione indipendente**.

→ Pre-corso: Test, lint e code review e GitHub (cos'è una pull request).

Per le PR grandi (almeno 10 file) serve anche una struttura in due passaggi:

1. **Un passaggio per file**: ogni file viene analizzato da solo, con la stessa attenzione.
2. **Un secondo passaggio**, con un prompt diverso, che controlla come i file interagiscono e riceve in ingresso i risultati del primo.

Perché due passaggi? Un unico passaggio su tanti file si stanca sui primi, applica criteri diversi da un file all'altro e non vede i problemi di integrazione: una funzione cambiata in un file che rompe il chiamante in un altro.

Un'analogia: per correggere un libro di 20 capitoli, prima un correttore rilegge ogni capitolo da solo (refusi, frasi poco chiare); poi un editor legge i riassunti di tutti i capitoli per verificare che la trama sia coerente (un personaggio morto al capitolo 3 non può ricomparire al 15).

---

7. Thinking, effort e parametri dei modelli recenti

Questa parte è più tecnica, ma è spesso una trappola nelle domande su migrazioni di modello. Una **migrazione di modello** è il passaggio di un'applicazione da un modello Claude più vecchio a uno più recente: alcune impostazioni che funzionavano prima possono non funzionare più.

→ Pre-corso: Richiesta HTTP. L'errore `400` significa "richiesta non valida": il server la rifiuta senza eseguirla.

Il **thinking** è la fase in cui Claude ragiona prima di rispondere. Nei materiali più vecchi si poteva fissare quanto ragionare con un numero (`budget_tokens`) e regolare la creatività con `temperature`. Sui modelli recenti questi parametri **danno errore `400`**: la richiesta viene rifiutata.

→ Pre-corso: Temperatura

Cosa si usa al loro posto:

- `thinking={"type": "adaptive"}`: il modello decide da solo quanto ragionare.
- `output_config.effort`: la profondità del lavoro, impostata in questo campo, non al livello principale. Il valore predefinito è `high`, tranne Opus 5.5 che usa `medium`.

Altre cose che sui modelli recenti non funzionano:

- **Prefill**: scrivere in anticipo l'inizio della risposta del modello per forzarne il formato. Dà errore `400`. Si usa `output_config.format` o un'istruzione nel system prompt.

Il **system prompt** è il testo di istruzioni generali che vale per tutta la conversazione. Il **thinking** è il ragionamento interno che Claude fa prima di rispondere.

Tabella riassuntiva per le migrazioni:

Prima (modelli vecchi)	Sui modelli recenti	Al suo posto
`budget_tokens`	Errore `400`	`thinking: {"type": "adaptive"}` e `output_config.effort`
`temperature`	Errore `400`	Istruzioni, esempi, output strutturato
Prefill della risposta	Errore `400`	`output_config.format` o istruzione nel system prompt

8. La cache dei prompt: non pagare due volte lo stesso testo

→ Pre-corso: Cache e Token e finestra di contesto (perché ogni token si paga).

Ogni volta che mandi una richiesta, Claude legge tutto il testo: strumenti, istruzioni, messaggi. Se la parte iniziale è sempre la stessa, il sistema può **riusarla** dalla memoria veloce, costa molto meno e risponde prima. È come preparare in anticipo la base di una ricetta che fai ogni giorno.

L'ordine è fisso: **strumenti** → **system prompt** → **messaggi**. Se cambi qualcosa all'inizio, si invalida tutto ciò che viene dopo.

Pensa a una fila di tessere del domino: la cache funziona finché il testo è identico dall'inizio. Appena una tessera è diversa, tutte quelle dopo vanno rielaborate (e ripagate).

Regola di progettazione: **prima ciò che non cambia mai, poi ciò che cambia**.

- Prima: system prompt fisso, strumenti in ordine stabile.
- Dopo: timestamp, numeri di pratica, la domanda dell'utente.

Trappole che invalidano la cache senza che te ne accorga:

- la data e l'ora inserite nel system prompt (cambiano a ogni richiesta);
- un JSON con le chiavi in ordine casuale;
- un insieme di strumenti che cambia da una richiesta all'altra;

- un ID di sessione messo nella parte iniziale.

Esempio d'esame: "*Abbiamo attivato la cache, ma il costo non scende.*" Prima cosa da cercare: qualcosa che cambia a ogni richiesta **all'inizio** del prompt, tipicamente la data e l'ora nel system prompt. La soluzione è spostarla nel messaggio dell'utente, dopo la parte fissa.

---

9. Le cinque cose da ricordare

1. Criteri espliciti: cosa rientra, cosa no, esempi di confine.
2. Lo schema garantisce la forma; la correttezza va controllata nel codice.
3. Dato assente = `null`, non nuovo tentativo.
4. Batch per il non urgente, sincrono per il bloccante. In batch, `custom_id` è la chiave.
5. Revisione in sessione indipendente, e per le PR grandi in più passaggi.

Per capire meglio

Quando una domanda parla di "migrazione di modello" e parametri che danno errore, pensa a `budget_tokens`, `temperature` e prefill: sono gli indizi. Quando parla di "dato mancante" e riprova, la risposta è `null`. Quando parla di "numeri che non tornano" con uno schema valido, la risposta è la convalida nel codice.

Tre ragionamenti tipici

"La revisione automatica segnala troppi problemi irrilevanti e gli sviluppatori la ignorano." Servono criteri espliciti per categoria (cosa rientra, cosa no, esempi di confine) e, per le categorie rumorose, disattivarle finché non sono migliorate. Non basta chiedere di "segnalare solo i problemi importanti".

"Il JSON estratto è sempre valido, ma a volte il totale non corrisponde alla somma delle voci." Lo schema funziona: il problema è di correttezza. Serve un controllo nel codice che confronti somma e totale e, in caso di differenza, rimandi a Claude l'errore preciso.

"Dobbiamo analizzare 20.000 contratti entro la settimana e ridurre i costi." Nessuno aspetta in tempo reale: Batch API, `custom_id` per ogni contratto, prova prima su un piccolo campione, reinvio solo delle richieste fallite.